

Pong 游戏改进

用函数重构 + 暂停/获胜/提示栏

姓 名：张轩基

学 号：1030425213

班 级：计科2502

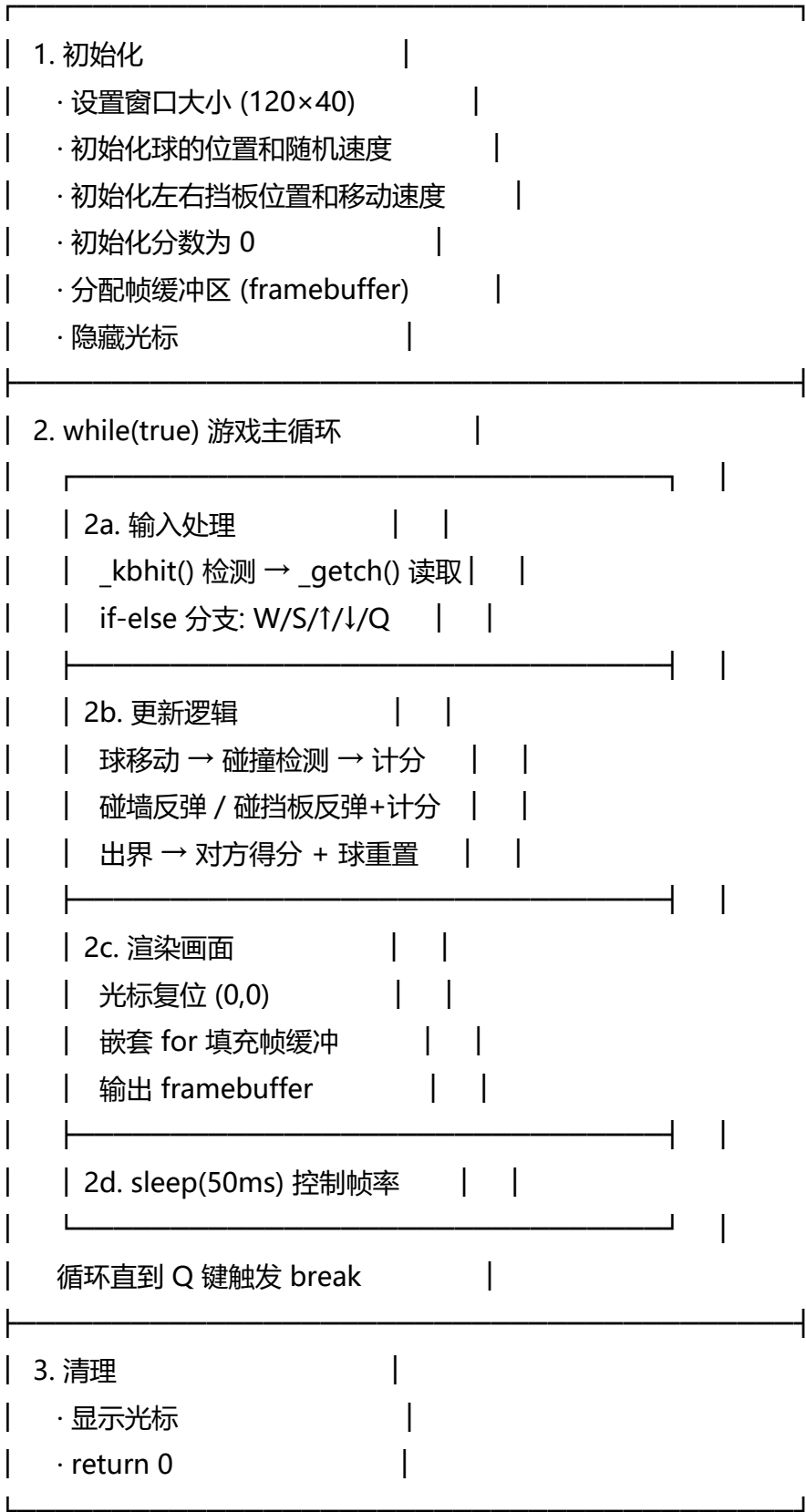
提交日期：2026.6.26

源代码： <https://github.com/JieMoJi/Island-survival>

改进 D (暂停) + A (5分获胜) + C (底部提示栏)

1. 原版 Pong 游戏流程分析

原版 Pong 游戏的主流程遵循经典的「游戏循环」模式：



2. 改进任务选择与说明

本次实验选择了以下三项改进，以 D（暂停功能）为主，A（5分获胜条件）和 C（底部提示栏）为辅：

【改进 D — 暂停功能】（主要改进）

按下 P 键可暂停/继续游戏。暂停时：

- 球的物理更新停止（跳过 `update_game` 函数调用）
- 挡板移动禁用（跳过正常操作的按键处理）
- 画面中央显示「游戏暂停」覆盖层
- 底部提示栏切换为暂停模式的操作说明

【改进 A — 获胜条件】

先得 5 分者获胜。新增以下逻辑：

- 每次计分后检查 `score1 >= 5` 或 `score2 >= 5`
- 满足条件时设置 `gameOver=true`, `winner=1`或`2`
- 画面中央显示获胜信息覆盖层
- 底部提示栏切换为「按 R 重新开始 / 按 Q 退出」

【改进 C — 底部提示栏】

在游戏画面底部增加了 2 行信息栏：

- 正常模式：显示 W/S（左挡板）、↑/↓（右挡板）、P（暂停）、Q（退出）以及获胜条件
- 暂停模式：显示 P（继续）、Q（退出）
- 结束模式：显示 R（重新开始）、Q（退出）
- 提示栏与游戏画面之间用分隔线隔开，视觉清晰

3. 函数设计与代码改动

本次改进的关键变化是：将原版全部写在 `main()` 中的代码拆分为 6 个独立函数，每个函数负责单一职责。以下是所有新增/修改的函数说明：

1. `handle_input()` — 输入处理函数

职责：统一处理所有键盘输入，根据游戏状态（正常/暂停/结束）决定响应方式。

参数：挡板位置引用、暂停状态引用、游戏结束状态引用、分数引用等

返回：bool (`true`=继续运行, `false`=退出程序)

新增逻辑：P键暂停切换、R键重新开始（仅`gameOver`时有效）、暂停/结束时禁用挡板操作

2. `update_game()` — 游戏逻辑更新函数

职责：更新球的位置、处理碰撞检测、计分、检测获胜条件。

参数：球的位置和速度引用、分数引用、gameOver和winner引用

新增逻辑：计分后检查 $score \geq 5 \rightarrow$ 设置 $gameOver = true$

3. render_frame() — 画面渲染函数

职责：填充帧缓冲（墙壁、球、挡板、分数），绘制覆盖层和底部信息栏。

新增逻辑：根据 paused/gameOver 状态绘制不同的覆盖层文字

4. draw_text_to_buffer() — 辅助函数：在帧缓冲指定位置写文字

职责：向帧缓冲指定坐标写入一段文字，用于覆盖层显示。

5. draw_centered_overlay() — 辅助函数：居中覆盖层

职责：在画面中央绘制多行文字框（用于暂停和游戏结束的提示）。

参数：帧缓冲引用、画面宽高、4行文字内容（空字符串表示跳过该行）

6. draw_info_bar() — 辅助函数：底部信息栏

职责：根据游戏状态（正常/暂停/结束）在画面底部绘制不同的操作提示。

参数：帧缓冲引用、画面宽高、paused和gameOver状态

7. reset_game() — 辅助函数：重置游戏状态

职责：将所有游戏变量恢复为初始值（分数归零、球重置、挡板归位、状态重置）。

参数：所有游戏状态变量的引用

函数调用关系：

main()

- |— handle_input() 每一帧调用，处理按键
- |— update_game() 仅当 !paused && !gameOver 时调用
- |— render_frame() 每一帧调用
- | |— draw_centered_overlay() 绘制暂停/获胜覆盖层
- | |— draw_info_bar() 绘制底部提示栏
- |— reset_game() 按 R 重新开始时调用

3.1 核心改动代码 — 输入处理（暂停逻辑）

```

// handle_input() 中的暂停逻辑 (改进 D 核心)
bool handle_input(..., bool& paused, bool& gameOver, ...) {
    char key;
    if (!_kbhit()) return true; // 无按键, 继续运行

    key = _getch();

    // 暂停切换 (游戏进行中才有效)
    if ((key == 'p' || key == 'P') && !gameOver) {
        paused = !paused;
        return true;
    }

    // 游戏结束后的 R 重新开始 (改进 A)
    if (gameOver) {
        if (key == 'r' || key == 'R') {
            reset_game(...); // 重置所有状态
        } else if (key == 'q' || key == 'Q') {
            return false; // 退出
        }
        return true;
    }

    // 暂停时只响应已处理的 P 和下面的 Q
    if (paused) {
        if (key == 'q' || key == 'Q') return false;
        return true; // 忽略挡板操作
    }

    // 正常操作: W/S 左挡板, 方向键右挡板, Q 退出
    if ((key == 'w' || key == 'W') && paddle1_y > paddle1_vec) { ... }
    ...
}

```

3.2 核心改动代码 — 获胜检测

```

// update_game() 中的获胜检测 (改进 A 核心)
void update_game(..., int& score1, int& score2,
                bool& gameOver, int& winner, ...) {
    ... // 球移动 + 碰撞检测 + 出界计分 (与原版相同)

    // 检查获胜条件: 先得 5 分者获胜
    if (score1 >= 5) {
        gameOver = true;
        winner = 1;
    } else if (score2 >= 5) {
        gameOver = true;
        winner = 2;
    }
}

```

3.3 核心改动代码 — 覆盖层与信息栏渲染

```

// render_frame() 中的覆盖层绘制 (改进 D + A)
void render_frame(..., bool paused, bool gameOver, int winner) {
    ... // 绘制墙壁、球、挡板、分数

    // 暂停覆盖层
    if (paused && !gameOver) {
        draw_centered_overlay(fb, WIDTH, HEIGHT,
            "┌──────────────────┐",
            "│  游戏暂停  │",
            "│  按 P 键继续...  │",
            "└──────────────────┘");
    }

    // 游戏结束覆盖层
    if (gameOver) {
        string winner_text =
            " PLAYER " + to_string(winner) + " 获胜! ";
        draw_centered_overlay(fb, WIDTH, HEIGHT,
            "┌──────────────────┐",
            "│  游戏结束  │",
            "│ " + winner_text + " │",
            "└──────────────────┘");
    }

    // 底部信息栏 (改进 C)
    draw_info_bar(fb, WIDTH, HEIGHT, paused, gameOver);
}

```

3.4 核心改动代码 — 底部信息栏

```

// draw_info_bar() 根据状态切换提示文字 (改进 C 核心)
void draw_info_bar(string& fb, int WIDTH, int HEIGHT,
    bool paused, bool gameOver) {
    string info;
    if (gameOver) {
        info = " [R] 重新开始 | [Q] 退出游戏";
    } else if (paused) {
        info = " [P] 继续游戏 | [Q] 退出游戏";
    } else {
        info = " [W/S] 左挡板 | [↑/↓] 右挡板 |"
            " [P] 暂停 | [Q] 退出 | 先得5分获胜! ";
    }
    // 将 info 写入帧缓冲最后一行
    int row = HEIGHT + 1;
    for (int x = 0; x < (int)info.size() && x < WIDTH; x++)
        fb[row * (WIDTH + 1) + x] = info[x];
    ...
}

```

3.5 主循环改动

```
// main() 中的游戏主循环 (改进后的简化结构)
int main() {
    ... // 变量声明 + 初始化

    bool paused = false;    // 新增: 暂停状态
    bool gameOver = false;  // 新增: 游戏结束状态
    int winner = 0;         // 新增: 获胜者编号

    bool running = true;
    while (running) {
        // 1. 处理输入 (含暂停/重新开始逻辑)
        running = handle_input(...);

        // 2. 更新逻辑 (暂停或结束时跳过)
        if (!paused && !gameOver) {
            update_game(...);
        }

        // 3. 渲染画面 (含覆盖层和信息栏)
        render_frame(...);

        // 4. 输出 + 延时
        cout << framebuffer;
        sleep_for(50ms);
    }
}
```

4. 遇到的问题与解决方法

问题 1：暂停时挡板仍能移动

现象：暂停后按 W/S/↑/↓ 键，挡板竟然还在动。

原因：原版的按键处理逻辑在 `handle_input()` 中先读取了按键，
暂停标记的设置和挡板操作在同一个函数的不同分支中，
但挡板移动的判断条件写在了暂停判断之前。

解决：调整 if-else if 的顺序——先判断暂停状态，暂停时跳过所有挡板操作。

重构后的 `handle_input()` 按照「P 键 → 游戏结束操作 → 暂停阻挡 → 正常操作」
的顺序组织分支，确保了暂停时所有游戏操作都被拦截。

问题 2：游戏结束后球仍在移动

现象：获胜弹窗显示了，但画面上的球还在飞，分数还在变化。

原因：原来的更新逻辑放在主循环中无条件执行，没有检查 `gameOver` 状态。

解决：在调用 `update_game()` 前增加条件判断：if (!paused && !gameOver)。

游戏结束或暂停时，物理更新被完全跳过，画面冻结在结束瞬间。

问题 3：重新开始后状态残留

现象：按 R 重新开始后，旧的「游戏结束」文字仍然显示在画面上。

原因：重置时只重置了分数和球的位置，没有把 `gameOver` 和 `winner` 重置。

解决：编写专门的 `reset_game()` 函数，一次性重置所有 10+ 个状态变量，
避免遗漏。函数将 `paused`、`gameOver`、`winner` 以及所有游戏数据统一归零。

问题 4：帧缓冲大小不足

现象：添加底部信息栏后，文字显示不完整或覆盖了底部墙。

原因：原版帧缓冲大小为 $HEIGHT * (WIDTH + 1)$ ，刚好容纳游戏画面。

新增的信息栏需要额外的行。

解决：扩展帧缓冲大小为 $(HEIGHT + 1 + INFO_H) * (WIDTH + 1)$ ，其中 `INFO_H=2`，
多出 2 行用于底部提示栏。同时修改渲染函数，将信息栏写在这两行中。

5. 实验总结与心得

通过本次实验，我完成了从「阅读代码 → 理解流程 → 函数拆分 → 功能扩展」的完整过程，
主要收获如下：

1. 函数拆分提升了代码的可维护性：

原版 Pong 的全部逻辑（初始化、输入、更新、渲染）都写在 `main()` 中，约 140 行。

改进后拆分为 7 个函数，`main()` 缩减到约 50 行，变成一个清晰的「初始化→循环→清理」骨架。

每个函数的职责单一明确，单独修改某项功能不再需要翻阅整个 main()。

2. 状态管理是游戏编程的核心：

引入 paused 和 gameOver 两个布尔变量后，游戏从「单状态（运行中）」变为「三状态（运行中/暂停/结束）」。状态变量的引入使得：

- 输入处理需要根据状态决定响应哪些按键
- 更新逻辑需要根据状态决定是否执行
- 渲染需要根据状态决定显示什么覆盖层

这种「状态驱动」的设计模式是游戏编程的基本范式。

3. 帧缓冲技术理解加深：

通过向帧缓冲写入覆盖层文字（而非简单的 cout），理解了为什么游戏使用帧缓冲而非逐行输出：它允许在渲染的最后阶段叠加 UI 元素（暂停提示、信息栏），而不会与游戏画面产生闪烁或错位。

4. 改进选择的心得：

选择 D+A+C 三项组合而非单独一项，因为这三项之间有自然的配合关系：

- 暂停（D）需要底部提示栏（C）来告知玩家当前状态和操作方式
- 获胜条件（A）使游戏有了「结束状态」，暂停（D）需要区分「运行中暂停」和「结束后暂停」
- 信息栏（C）在三种状态下显示不同内容，与 A 和 D 共同构成完整的状态反馈系统

三项改进协同形成了一个有头有尾、操作清晰的完整游戏体验。

原版 vs 改进版 对比

维度	原版 Pong	改进版 Pong
代码结构	全部在 main() 中，约 140 行	7 个函数，main() 约 50 行
函数数量	main() + 2 个平台函数	main() + 2 平台函数 + 7 个业务函数
游戏状态	单状态（运行中）	三状态（运行/暂停/结束）
暂停功能	无	P 键切换，画面冻结 + 覆盖层
获胜条件	无（无限循环）	先得 5 分获胜，显示获胜者
重新开始	无（只能退出重进）	R 键一键重置所有状态
操作提示	无	底部 2 行动态提示栏，随状态切换
输入处理	if-else 分支	函数化，按游戏状态分层响应